EXERCISE 08: NAIVE BAYES CLASSIFICATION WITH LIVE USER INPUT

College Data Science Laboratory Course — Practical Examination Study Guide
Source Code File: `e08_naive_bayes.py` | **Repository:** `https://github.com/akhilkvu6/ds` | **Algorithm Family:** Supervised Learning — Probabilistic Classification

1. Program Identification and Aim

- **File Name:** `e08_naive_bayes.py`
- **Academic Aim:** To construct an end-to-end interactive machine learning pipeline utilizing the **Gaussian Naive Bayes algorithm** on the classical **Iris Flower Dataset**, train the model with an explicit 80:20 training-testing split, evaluate classification percentage accuracy on the testing partition, and implement interactive terminal input parsing that enables an external user to feed arbitrary continuous real-world flower measurements into the trained model and instantly obtain the predicted biological class name (*setosa*, *versicolor*, or *virginica*).
- **Key Differences from Exercise 06 (Standard Naive Bayes):**
  1. **Train/Test Partitioning:** Exercise 06 relies on scikit-learn's default split (75% train / 25% test), whereas Exercise 08 explicitly sets `train_size=0.8` (80% train / 20% test).
  2. **Evaluation Metric Display:** Exercise 06 outputs a multidimensional Confusion Matrix and an extensive Classification Report (Precision, Recall, F1-Score, Support). Exercise 08 computes and prints a clean, single percentage metric using `accuracy_score(y_test, result) * 100`.
  3. **Live Interactive Prediction:** Exercise 06 executes entirely autonomously on static test samples. Exercise 08 features a live dynamic prompt (`input(...)`) with whitespace splitting, float conversion, 2D matrix encapsulation, and target class name mapping.
- **Core Output:** Terminal prints the test set percentage accuracy followed by an interactive prompt asking for 4 flower measurements, and displays the mapped predicted flower species array.

2. Prerequisites and Conceptual Foundations

- **Theoretical Background:** Supervised Multiclass Classification, Conditional Probability, Bayes' Theorem, Gaussian Probability Density Function (Normal Distribution assumption for continuous features), Class Conditional Independence assumption.
- **Required Python Modules:**
  - `sklearn.datasets`: Provides `load_iris()` to supply standard Iris benchmark data.
  - `sklearn.naive_bayes`: Provides `GaussianNB()` implementing Gaussian Naive Bayes.
  - `sklearn.model_selection`: Provides `train_test_split()` for dataset partitioning.
  - `sklearn.metrics`: Provides `accuracy_score()` to compare predicted test labels against ground truth.
- **System/Environment Assumptions:** Python 3.8+ with `scikit-learn`, `numpy`. No local CSV or TXT dependencies; dataset is loaded directly from memory via scikit-learn builtins. Interactive standard input (keyboard) is required for the second phase of execution.

3. Complete Exam-Ready Algorithm (START ... STOP Format)

ALGORITHM: Gaussian Naive Bayes Classification with Live User Input

Step 1: START
Step 2: [Import Required Modules]
Import `datasets` and `naive_bayes` from `sklearn`.
Import `train_test_split` from `sklearn.model_selection`.
Import `accuracy_score` from `sklearn.metrics`.
Step 3: [Load Iris Dataset]
Invoke `datasets.load_iris()` and store the dataset object in `iris`.
Step 4: [Extract Feature and Target Arrays]
Assign `iris.data` (150 samples × 4 features) to `x`.
Assign `iris.target` (150 class labels: 0, 1, 2) to `y`.
Step 5: [Instantiate Gaussian Naive Bayes Classifier]
Create model object `classifier = naive_bayes.GaussianNB()`.
Step 6: [Partition Dataset into Training and Testing Sets]
Execute `train_test_split(x, y, train_size=0.8)` to reserve 80% (120 rows) for `x_train, y_train` and 20% (30 rows) for `x_test, y_test`.
Step 7: [Train the Classifier]
Fit model parameters by executing `classifier.fit(x_train, y_train)`, which computes per-class priors, means, and variances for all 4 features.
Step 8: [Predict Test Partition Labels]
Generate predictions for testing data: `result = classifier.predict(x_test)`.
Step 9: [Calculate and Display Test Accuracy]
Compute accuracy percentage: `accuracy = accuracy_score(y_test, result) * 100`.
Print `Accuracy is {accuracy}`.
Step 10: [Accept and Parse Live User Input]
Display interactive prompt `"Enter " + iris.feature_names`.
Split user input string by whitespace, convert each token into a floating-point number using list comprehension, and wrap the list inside an outer list to construct a 2D array: `features = [[float(i) for i in input(...).split()]]`.
Step 11: [Predict Species for User-Provided Measurements]
Evaluate `result = classifier.predict(features)`.
Step 12: [Display Target Class Name]
Map numeric class index (0, 1, or 2) back to human-readable species name via `iris["target_names"][result]` and display on terminal.
Step 13: STOP

4. Complete Theory, Formulas, and Numerical Worked Example

4.1 Bayes' Theorem and Gaussian Likelihood

Naive Bayes classifiers are founded upon Bayes' Rule of conditional probability. For an observation vector $\mathbf{x} = (x_1, x_2, \dots, x_d)$ and class label $C_k \in \{0, 1, 2\}$:

$$P(C_k \mid \mathbf{x}) = \frac{P(C_k) \cdot P(\mathbf{x} \mid C_k)}{P(\mathbf{x})} = \frac{P(C_k) \prod_{j=1}^d P(x_j \mid C_k)}{\sum_{m} P(C_m) \prod_{j=1}^d P(x_j \mid C_m)}$$

The core "naive" assumption is that all continuous features x_1, x_2, x_3, x_4 are conditionally independent given class C_k. For continuous measurements, $P(x_j \mid C_k)$ is modelled using the Gaussian (Normal) probability density function:

$$P(x_j \mid C_k) = \frac{1}{\sqrt{2\pi \sigma_{kj}^2}} \exp\left(-\frac{(x_j - \mu_{kj})^2}{2\sigma_{kj}^2} \right)$$

Where:

- μ_{kj}: Empirical mean of feature j across all training observations belonging to class C_k.
- σ_{kj}^2: Empirical variance of feature j across training observations of class C_k.
- $P(C_k)$: Class prior probability, estimated as $\frac{N_k}{N_{\text{total}}}$.

4.2 Step-by-Step Worked Numerical Example

Worked Example: Suppose a user inputs measurements for Petal Length (x) to classify between *Setosa* (C_1) and *Versicolor* (C_2). Assume equal priors $P(C_1) = P(C_2) = 0.5$.

Learned Parameters from Training Set:
• Setosa (C_1): Mean $\mu_1 = 1.46\text{ cm}$, Variance $\sigma_1^2 = 0.03\text{ cm}^2$
• Versicolor (C_2): Mean $\mu_2 = 4.26\text{ cm}$, Variance $\sigma_2^2 = 0.22\text{ cm}^2$

User Input Test Measurement: $x = 1.5\text{ cm}$ (User entered decimal measurement)

1. Calculate Gaussian Likelihood for Setosa (C_1): $$P(x=1.5 \mid C_1) = \frac{1}{\sqrt{2\pi (0.03)}} \exp\left(-\frac{(1.5 - 1.46)^2}{2(0.03)}\right) = \frac{1}{0.434} \exp\left(-\frac{0.0016}{0.06}\right) = 2.304 \times \exp(-0.0267) \approx 2.24$$
2. Calculate Gaussian Likelihood for Versicolor (C_2): $$P(x=1.5 \mid C_2) = \frac{1}{\sqrt{2\pi (0.22)}} \exp\left(-\frac{(1.5 - 4.26)^2}{2(0.22)}\right) = \frac{1}{1.176} \exp\left(-\frac{7.6176}{0.44}\right) = 0.850 \times \exp(-17.31) \approx 2.75 \times 10^{-8}$$
3. Compute Unnormalized Posterior (Prior × Likelihood):
• Setosa Posterior Score: $0.5 \times 2.24 = 1.12$
• Versicolor Posterior Score: $0.5 \times 2.75 \times 10^{-8} = 1.375 \times 10^{-8}$

Conclusion: Since $1.12 \gg 1.375 \times 10^{-8}$, the model assigns class label 0 (*setosa*).

4.3 In-Depth Analysis of User-Input Parsing Line

The user-input parsing line in `e08_naive_bayes.py` is an essential exam topic:

```
features = [
   [float(i) for i in input(
       f"Enter {iris.feature_names}"
   ).split()]
]
```

- `input(f"Enter {iris.feature_names}")`: Displays the prompt showing the 4 required feature names: `['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']` and waits for the user to type values separated by spaces (e.g. `5.1 3.5 1.4 0.2`).
- `.split()`: Breaks the input string by any whitespace characters into a Python list of individual string tokens: `['5.1', '3.5', '1.4', '0.2']`.
- `float(i) for i in ...`: Converts every string token into a 64-bit IEEE floating-point number. **Critical Note:** Using `int(i)` will crash with a `ValueError` whenever decimal values like `5.1` or `1.4` are entered. Real Iris flower measurements are continuous decimals; therefore, `float(i)` is strictly required.
- `[[ ... ]]` (Double Enclosing Square Brackets): Construct a 2-dimensional list of shape `(1, 4)`. Scikit-learn estimators mandate that the `predict()` method receives a 2D matrix where axis 0 is the batch of samples ($n_samples=1$) and axis 1 is the features ($n_features=4$). Passing a 1D list `[5.1, 3.5, 1.4, 0.2]` triggers a `ValueError: Expected 2D array, got 1D array instead`.
- `iris["target_names"][result]`: `result` is an array containing the predicted integer (e.g. `array([0])`). Indexing `iris["target_names"]` (which is `np.array(['setosa', 'versicolor', 'virginica'])`) with `result` maps index `0` to `['setosa']`.

5. Complete Original Source Code and Block-by-Block Walkthrough

5.1 Verbatim Source Code (`e08_naive_bayes.py`)

```
# Exercise 8: Naive Bayes

# Question 1: Program to implement Naive Bayes Algorithm using Iris dataset and find the accuracy.

from sklearn import datasets
from sklearn import naive_bayes
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

iris = datasets.load_iris()

x, y = iris.data, iris.target

classifier = naive_bayes.GaussianNB()

x_train, x_test, y_train, y_test = train_test_split(
    x, y, train_size=0.8
)

classifier.fit(x_train, y_train)

result = classifier.predict(x_test)

accuracy = accuracy_score(y_test, result) * 100

print(f"Accuracy is {accuracy}")

features = [
    [float(i) for i in input(
        f"Enter {iris.feature_names}"
    ).split()]
]

result = classifier.predict(features)

print(iris["target_names"][result])
```

5.2 Block-by-Block Walkthrough

Lines	Code Block	Detailed Execution Explanation
1–4	Header Comments	Documents laboratory exercise identity, topic (Naive Bayes), and problem statement (Iris classification with accuracy).
5–8	Module Imports	Imports `datasets` (Iris data provider), `naive_bayes` (classifier module), `train_test_split` (validation partitioning), and `accuracy_score` (metric function).
10	Data Loading	Loads the Bunch dictionary object representing the Iris dataset containing 150 instances across 3 flower species.
12	Feature/Target Extraction	Assigns the 2D feature matrix (150 rows, 4 columns) to `x` and the 1D target vector of class labels (0, 1, 2) to `y`.
14	Model Instantiation	Instantiates `GaussianNB()` object named `classifier`, ready to fit mean and variance per feature.
16–18	Train/Test Split	Splits 150 instances into 80% training set (120 samples) and 20% test set (30 samples) using `train_size=0.8`.
20	Model Fitting	Trains the classifier on `x_train` and `y_train` by calculating the mean (μ_{kj}) and variance (σ^2_{kj}) for all features for each class.
22	Test Prediction	Feeds `x_test` (30 samples) through the trained classifier, computing posterior probabilities and assigning class labels to `result`.
24–26	Accuracy Evaluation	Calculates the proportion of correctly classified test instances via `accuracy_score(y_test, result)`, multiplies by 100 to get a percentage, and prints the result.
28–32	Interactive User Input	Prompts user with feature names, accepts raw string input, splits by whitespace, converts values to `float`, and creates a 2D matrix `[[f1, f2, f3, f4]]`.
34	User Sample Prediction	Calls `classifier.predict(features)` on the user-provided sample and reuses the variable name `result` to store the predicted integer label.
36	Target Name Mapping & Display	Indexes `iris["target_names"]` using the predicted integer array to print the species name in English (e.g. `['setosa']`).

6. Complete Function, Parameter, and Syntax Reference

6.1 Comprehensive 10-Point Technical Breakdown

1. **`datasets.load_iris()`:** Returns a dictionary-like `Bunch` object containing `data` (150 × 4 NumPy array), `target` (150 integer labels), `target_names` (species names), and `feature_names` (attribute names).
2. **`iris.data` and `iris.target`:** NumPy ndarrays. `iris.data.shape == (150, 4)` with columns Sepal Length, Sepal Width, Petal Length, Petal Width. `iris.target.shape == (150,)` with integer encodings 0 (setosa), 1 (versicolor), 2 (virginica).
3. **`naive_bayes.GaussianNB()`:** Classifier implementing the Gaussian Naive Bayes algorithm. Parameters include `priors=None` (class priors automatically calculated from training set proportions) and `var_smoothing=1e-9` (numerical stability constant added to variances).
4. **`train_test_split(x, y, train_size=0.8)`:** Partitions data arrays into training and testing subsets. When `train_size=0.8` is provided without `test_size`, the remaining 20% is assigned to test. Returns four subsets in order: `x_train, x_test, y_train, y_test`.
5. **`classifier.fit(x_train, y_train)`:** Estimates parameters from training data. For GaussianNB, this computes the mean array `classifier.theta_` and variance array `classifier.var_` for each class and feature.
6. **`classifier.predict(X)`:** Evaluates posterior probability $P(C_k \mid \mathbf{x}) \propto P(C_k) \prod P(x_j \mid C_k)$ and returns the class with maximum posterior probability: $\hat{y} = \arg\max_k P(C_k \mid \mathbf{x})$. Requires a 2D array of shape `(n_samples, n_features)`.
7. **`accuracy_score(y_true, y_pred)`:** Calculates fraction of correct predictions: $\frac{1}{N} \sum_{i=1}^N \mathbb{I}(y_i = \hat{y}_i)$. Multiplied by 100 to yield a percentage.
8. **`input(prompt)`:** Built-in Python function that pauses execution, writes prompt string to standard output, and reads one line from standard input as a string.
9. **`str.split()`:** String method that partitions a string on whitespace boundaries (spaces, tabs, newlines), discarding consecutive whitespace, returning a list of string tokens.
10. **`iris["target_names"][result]`:** Advanced NumPy array indexing. `iris["target_names"]` is `np.array(['setosa', 'versicolor', 'virginica'])`. Passing integer array `result` (e.g. `[0]`) retrieves `array(['setosa'], dtype='.`

6.2 Technical Coverage Checklist

Technical Concept / Item	Line Number	Status	Exam Relevance
Module Import Hierarchy	Lines 5–8	✓ Verified	Must know exact import paths from sklearn
Bunch Object Attribute Access	Lines 10, 12, 30, 36	✓ Verified	`.data`, `.target`, `.feature_names`, `["target_names"]`
Explicit 80:20 Partitioning	Lines 16–18	✓ Verified	`train_size=0.8` parameter syntax
Model Fitting & Scoring	Lines 20, 24	✓ Verified	`fit()` then `accuracy_score() * 100`
List Comprehension with Float Conversion	Lines 28–32	✓ Verified	Essential syntax: `[float(i) for i in ...]`
2D Array Requirement for `predict()`	Lines 28, 34	✓ Verified	Outer brackets `[[ ... ]]` to prevent dimension error
Target Name Array Indexing	Line 36	✓ Verified	Maps integer back to English biological species

7. Step-by-Step Execution Walkthrough and Variable State Trace

Step	Line	Action / Expression	Variable State / Output
1	10	`load_iris()`	`iris`: Bunch object with 150 samples and metadata.
2	12	Extract `x` and `y`	`x.shape == (150, 4)`, `y.shape == (150,)`
3	14	Instantiate model	`classifier`: `GaussianNB()` instance initialized.
4	16–18	`train_test_split(train_size=0.8)`	`x_train.shape == (120, 4)`, `x_test.shape == (30, 4)`
5	20	`classifier.fit(x_train, y_train)`	`classifier.theta_.shape == (3, 4)` (means), `classifier.var_.shape == (3, 4)`
6	22	`classifier.predict(x_test)`	`result.shape == (30,)` (predictions for test partition)
7	24–26	Compute & display accuracy	`accuracy`: float (typically `93.33333333333333` to `100.0`)
8	29–31	Prompt user & parse input	Prompts: `Enter ['sepal length (cm)', ...]`. User types: `5.1 3.5 1.4 0.2`. `features`: `[[5.1, 3.5, 1.4, 0.2]]` (shape `(1, 4)`)
9	34	`classifier.predict(features)`	`result`: `array([0])`
10	36	Print mapped target name	Terminal displays: `['setosa']`

8. Output and Practical Interpretation

8.1 Sample Terminal Execution Output

```
Accuracy is 96.66666666666667
Enter ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']5.1 3.5 1.4 0.2
['setosa']
```

8.2 Additional Live Test Cases

Test Case	Terminal Input Typed by User	Actual Predicted Output	Biological Reason
1. Typical Setosa	`5.1 3.5 1.4 0.2`	`['setosa']`	Petal length < 2 cm and petal width < 0.5 cm uniquely characterize Setosa.
2. Typical Versicolor	`6.0 2.8 4.5 1.4`	`['versicolor']`	Intermediate petal length (4–5 cm) and moderate width (1.2–1.6 cm).
3. Typical Virginica	`6.7 3.1 5.6 2.4`	`['virginica']`	Large petals (length > 5.2 cm, width > 2.0 cm) identify Virginica.
4. Boundary Case	`5.9 3.0 4.8 1.8`	`['versicolor']` or `['virginica']`	Close to boundary; Gaussian likelihood determines higher probability class.

8.3 Practical Interpretation of Results

- **Accuracy Score:** Out of 30 test samples, the Gaussian Naive Bayes model typically misclassifies only 0 to 2 samples, achieving 93.3% to 100.0% test accuracy. Because no fixed `random_state` is specified in `train_test_split()`, accuracy may vary slightly across runs.
- **Continuous User Input:** Real-world biometric data is continuous. By utilizing `float()`, any realistic measurement containing decimals is correctly evaluated.

9. Exam Preparation: Lab Record Answer and Model Questions

9.1 Lab Record Standard Format (Write in 1 Page)

> **EXERCISE 8: NAIVE BAYES CLASSIFICATION WITH USER INPUT**
>
> **AIM:** To write a Python program implementing the Gaussian Naive Bayes classification algorithm on the Iris dataset, evaluate test accuracy, and classify interactive user-entered flower measurements.
>
> **ALGORITHM:**
> 1. Import `datasets`, `naive_bayes`, `train_test_split`, and `accuracy_score`.
> 2. Load Iris dataset via `datasets.load_iris()` and split into features `x` and labels `y`.
> 3. Initialize `GaussianNB()` classifier.
> 4. Split data into 80% train and 20% test subsets using `train_test_split(train_size=0.8)`.
> 5. Fit the classifier on training data using `classifier.fit(x_train, y_train)`.
> 6. Predict test labels `classifier.predict(x_test)` and calculate accuracy percentage.
> 7. Accept 4 feature measurements from user, convert to float 2D array, predict class, and display mapped target name.
>
> **RESULT:** The Gaussian Naive Bayes model was successfully built, trained, and tested. The model achieved an accuracy of 96.67% and successfully predicted species for interactive user inputs.

9.2 Common Exam Questions and Model Answers

- **2-Mark Question:** Why is Naive Bayes called "naive"?
  Answer: It is called "naive" because it makes the simplifying assumption that all input features are mutually independent given the class label, which is rarely strictly true in real-world data.
- **3-Mark Question:** Why is `float(i)` necessary instead of `int(i)` in the input statement of `e08_naive_bayes.py`?
  Answer: Iris flower measurements are continuous decimal numbers (e.g., 5.1, 3.5). The `int()` function throws a `ValueError` when parsing a string containing a decimal point. Using `float(i)` properly parses real numbers.
- **5-Mark Question:** Explain why double square brackets `[[ ... ]]` are used when storing user input in `features`.
  Answer: Scikit-learn's `predict()` method strictly requires a two-dimensional array of shape `(n_samples, n_features)`. For a single sample with 4 features, the shape must be `(1, 4)`. If single brackets `[ ... ]` were used, the list would be 1D of shape `(4,)`, causing scikit-learn to throw: `ValueError: Expected 2D array, got 1D array instead. Reshape your data either using array.reshape(-1, 1) if your data has a single feature or array.reshape(1, -1) if it contains a single sample.` The outer brackets construct the required 2D structure.

C. Predict-the-Output Questions

1. **Question:** What is the output of `iris["target_names"][np.array([0])]`?
   Answer: `array(['setosa'], dtype='.`
2. **Question:** If user inputs `5.1 3.5 1.4 0.2`, what species is printed?
   Answer: `['setosa']`.

D. Identify-the-Function Questions

1. **Question:** Which built-in Python function prompts for and reads text input from the console?
   Answer: `input(prompt)`.
2. **Question:** Which string method splits an input string on whitespace boundaries into a list of substrings?
   Answer: `str.split()`.
3. **Question:** Which type converter must be used so that decimal numbers can be parsed without error?
   Answer: `float()`.

10. Comprehensive Viva Voce Questions and Answers (16 Exam Questions)

Q1: What mathematical theorem forms the foundation of the Naive Bayes algorithm?
Answer: Bayes' Theorem of conditional probability: $P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)}$, where $P(A \mid B)$ is posterior probability, $P(B \mid A)$ is likelihood, $P(A)$ is prior, and $P(B)$ is marginal likelihood.

Q2: Why does this program use GaussianNB rather than MultinomialNB or BernoulliNB?
Answer: Iris features (sepal length, sepal width, petal length, petal width) are continuous real numbers assumed to follow a normal (Gaussian) distribution. MultinomialNB is suited for discrete word counts, and BernoulliNB is for binary (0/1) features.

Q3: What parameters does GaussianNB compute and store during the `fit()` phase?
Answer: For each class C_k and feature x_j, it computes and stores the class prior $P(C_k)$, the feature mean μ_{kj} (stored in `theta_`), and the feature variance σ_{kj}^2 (stored in `var_`).

Q4: What is the train/test split ratio configured in this program?
Answer: 80% training (120 samples) and 20% testing (30 samples), established via `train_test_split(x, y, train_size=0.8)`.

Q5: What happens if you pass an integer value like `5` when prompted for user input?
Answer: `float("5")` evaluates successfully to `5.0` without error. Python's `float()` constructor handles both integer and decimal string inputs seamlessly.

Q6: What happens if you use `int(i)` and enter `5.1 3.5 1.4 0.2`?
Answer: Python crashes immediately with a `ValueError: invalid literal for int() with base 10: '5.1'`, because `int()` cannot parse strings containing decimal points.

Q7: Why does `split()` without arguments work best for reading multiple numbers?
Answer: `split()` defaults to splitting by any whitespace (spaces, tabs, newlines) and automatically ignores multiple consecutive spaces, making user typing error-tolerant.

Q8: What does `classifier.predict(features)` return?
Answer: It returns a 1D NumPy array containing the predicted integer class index for the supplied sample, such as `array([0])` for Setosa.

Q9: How does `iris["target_names"][result]` produce the species string?
Answer: `iris["target_names"]` contains `array(['setosa', 'versicolor', 'virginica'])`. Using `result` (which contains `[0]`) as an index returns an array containing the string at index 0, which is `array(['setosa'])`.

Q10: What is the difference in output between Exercise 06 and Exercise 08?
Answer: Exercise 06 outputs a Confusion Matrix and a detailed Classification Report (precision, recall, f1-score). Exercise 08 outputs an accuracy percentage and executes an interactive live user prediction.

Q11: Why is there no `random_state` parameter passed to `train_test_split()` in this program?
Answer: Without `random_state`, the random number generator is seeded non-deterministically (from system time), meaning the training and testing sets vary slightly across successive executions, leading to slight variations in accuracy (between 90% and 100%).

Q12: Does Naive Bayes require feature scaling or normalization?
Answer: No. Gaussian Naive Bayes computes mean and variance for each feature independently. Scale differences between features do not affect likelihood calculations, unlike distance-based models like KNN or K-Means.

Q13: What is "zero frequency problem" in Naive Bayes and how does GaussianNB handle it?
Answer: If a categorical feature value never occurs with a class in the training data, its conditional probability becomes zero, which zeros out the entire posterior product. In GaussianNB, variance smoothing (`var_smoothing=1e-9`) prevents division by zero or zero probability densities.

Q14: Can GaussianNB handle multi-class classification natively?
Answer: Yes, natively. Bayes' theorem computes posterior probabilities for any number of classes $k \in \{1, \dots, K\}$ without requiring One-vs-Rest or One-vs-One transformations.

Q15: What is the computational complexity of training and predicting with Gaussian Naive Bayes?
Answer: Training is $\mathcal{O}(N \cdot d)$ (computing mean and variance for N samples and d features). Prediction is $\mathcal{O}(K \cdot d)$ for K classes and d features. It is one of the fastest classification algorithms.

Q16: How would you modify this program to accept multiple user input rows at once?
Answer: By constructing a 2D list with multiple sub-lists: `features = [[5.1, 3.5, 1.4, 0.2], [6.2, 2.9, 4.3, 1.3]]`, `classifier.predict(features)` would return an array with multiple predictions: `array([0, 1])`.

Top 5 High-Priority Viva Questions (Last-Minute Refresh)

1. Why must we use `float(i)` instead of `int(i)` on user input?
Answer: Iris measurements are real numbers with decimal points (e.g. 5.1). Using `int()` raises a fatal `ValueError`.

2. Why are double square brackets `[[ ... ]]` required around user features?
Answer: Scikit-learn's `predict()` requires a 2D array of shape `(n_samples, n_features) = (1, 4)`; a 1D list causes a shape dimension error.

3. How does `iris["target_names"][result]` display the flower name?
Answer: `result` is an array containing the predicted integer (0, 1, or 2), which indexes the `target_names` array to return the corresponding species string.

4. What is the train/test split in `e08_naive_bayes.py`?
Answer: 80% train and 20% test, configured via `train_size=0.8` in `train_test_split()`.

5. How does Naive Bayes compute probabilities for continuous features?
Answer: Using the Gaussian Probability Density Function evaluated using the class feature mean μ and variance σ^2 learned during fitting.

11. Common Mistakes, Pitfalls, and Debugging Guide

Common Error / Pitfall	Root Cause	How to Fix / Debug
`ValueError: invalid literal for int() with base 10: '5.1'`	Using `int(i)` instead of `float(i)` when parsing decimal Iris measurements from terminal input.	Change `int(i)` to `float(i)` in the list comprehension on Line 29.
`ValueError: Expected 2D array, got 1D array instead`	Omitting the outer square brackets when wrapping the list of floats: `features = [float(i) for ...]`.	Add outer square brackets to create a 2D list: `features = [[float(i) for ...]]`.
`IndexError: list index out of range`	User entered fewer than 4 values separated by spaces (e.g. entered only 3 numbers).	Ensure exactly 4 whitespace-separated numeric values are provided at the terminal prompt.
Variable shadowing of `result`	`result` is first used for `x_test` predictions (Line 22) and reused for user input prediction (Line 34).	Harmless in this script because accuracy calculation finishes before the variable is overwritten, but in an exam use distinct names like `y_pred` and `user_pred` if requested.
Forgetting `* 100` on accuracy	`accuracy_score()` returns a decimal fraction between 0.0 and 1.0 (e.g. `0.9666`).	Multiply by 100 to display standard percentage format: `accuracy_score(...) * 100`.

12. Memory Tricks, Mnemonics, and Last-Minute Rapid Revision

12.1 Seven-Step Coding Flow Mnemonic: "L-S-M-F-P-A-I"

- **L — Load:** `iris = datasets.load_iris()` → `x, y = iris.data, iris.target`
- **S — Split:** `x_train, x_test, y_train, y_test = train_test_split(x, y, train_size=0.8)`
- **M — Model:** `classifier = naive_bayes.GaussianNB()`
- **F — Fit:** `classifier.fit(x_train, y_train)`
- **P — Predict:** `result = classifier.predict(x_test)`
- **A — Accuracy:** `accuracy = accuracy_score(y_test, result) * 100`
- **I — Input:** `features = [[float(i) for i in input(...).split()]]` → `predict` → `print(iris["target_names"][result])`

12.2 Memorizing the User-Input Line

> **Mental Template: Double-Bracket Float Comprehension**
> `features = [[float(i) for i in input(prompt).split()]]`
> - *Prompt:* `f"Enter {iris.feature_names}"`
> - *Break:* `.split()`
> - *Convert:* `float(i)`
> - *Wrap:* `[[ ... ]]`

13. Cross-Program Comparison Table (Programs 05 to 10)

The table below provides a rigorous cross-program comparative reference across all machine learning programs in the laboratory curriculum, highlighting learning paradigms, dataset characteristics, data splitting strategies, core algorithms, evaluation metrics, and diagnostic visualizations:

Program	Learning Paradigm	Dataset Used	Data Partitioning	Core Algorithm & Parameters	Evaluation Metrics	Diagnostic Visualizations
e05_knn.py	Supervised Classification	Iris Dataset (150 × 4)	`test_size=0.25` (75:25 split)	K-Nearest Neighbors (`n_neighbors=5`)	`accuracy_score()`	2 Subplots: Predicted vs Actual classification on test samples
e06_naive_bayes.py	Supervised Classification	Iris Dataset (150 × 4)	`test_size=0.3, random_state=40` (70:30)	Gaussian Naive Bayes (`GaussianNB()`)	`accuracy_score()` & `classification_report()`	None (Structured terminal classification report)
e07_decision_tree.py	Supervised Classification	Iris Dataset (150 × 4)	`test_size=0.3` (70:30 split)	Decision Tree (`criterion="entropy"` , `max_depth=4` , `min_samples_leaf=2`)	`classification_report()`	Full visual flowchart tree via `tree.plot_tree(filled=True)`
e08_naive_bayes.py	Supervised Classification	Iris Dataset (150 × 4)	`train_size=0.8` (80:20 split)	Gaussian Naive Bayes (`GaussianNB()`)	`accuracy_score() * 100` (%)	None (Live interactive console input parsing with `float` & species mapping)
e09_kmeans.py	**Unsupervised Clustering**	Synthetic Blobs (300 × 2)	None (Full unlabeled dataset)	K-Means Clustering (`n_clusters=5` , `random_state=42` , `n_init="auto"`)	Inertia / Cluster assignments (`labels_`)	2D Scatter plot with `c=labels, cmap="viridis"` and centroids marked with red 'x'
e10_multiple_linear_regression.py	**Supervised Regression**	Diabetes Dataset (Features 0 & 2)	Slicing (`[:-20]` train, `[-20:]` test)	Multiple Ordinary Least Squares (`LinearRegression()`)	Mean Squared Error (MSE) & R^2 Score	Figure 1: Feature vs Target subplots; Figure 2: Actual vs Predicted fit line